Dr. D. Y. Patil Educational Federation's
Dr. D. Y. PATIL COLLEGE OF ENGINEERING & INNOVATION
Survey No. 27/A/1/2C, Varale Campus, Near Talegaon Railway station,
Tal. Maval, Dist. Pune 410 507

PCC-203-COM: Operating Systems

Unit 1:Introduction to Operating System

Prof. Dnyanesh Gaikwad
Assistant Professor
Department of Computer Engineering

TG: SPPU_FE_ENGG

PDF BY VL







✨ WHY JOIN ?

🏆 Trusted by 80,000+ Engineering Students
🔔 Stay Updated with latest university news
🚀 Make your engineering journey easy with us

Best Regards, Vishwajeet Londhe

© 2026 Vishwajeet Londhe. All Rights Reserved.

Course Objectives

1. To understand the fundamental concepts, types, and structures of Operating Systems.
2. To understand and analyze process management concepts.
3. To identify and solve concurrency and deadlock in the operating system.
4. Explore the various techniques of memory management.
5. Understand I/O management, disk scheduling, and file systems.

TG. SPPU_FE_ENGG

Course Outcomes

Upon successful completion of this course, students will be able to:

CO1: **Analyze** the fundamentals of Operating Systems, including types, structures, system calls,and basic Linux commands.

CO2: **Apply** process scheduling and synchronization to optimize CPU utilization in modern operating systems.

CO3: **Identify** the mechanism for dealing with deadlocks and concurrency concerns.

CO4: **Apply** techniques of memory management to solve memory management problems

CO5: **Illustrate** I/O and file management policies.

TG:SPPU_FE_ENGG

Contents

- **Basics of Operating Systems:**
- Objectives & Functions
- Evolution of OS
- Types of Operating Systems
- OS Service
- System Calls: Introduction
- Types of System Calls
- **OS structure:** Layered Approach
- Monolithic
- Microkernel Operating Systems
- **Introduction to Linux OS:**
- Components of Linux system
- Basic Shell commands
- **Case studies:**
  1. Automating User and File Management in Linux using Shell Script
  2. Demonstrating Different OS Structures (Monolithic, Layered, Microkernel OS)

TG-SPPU_FE_ENGG

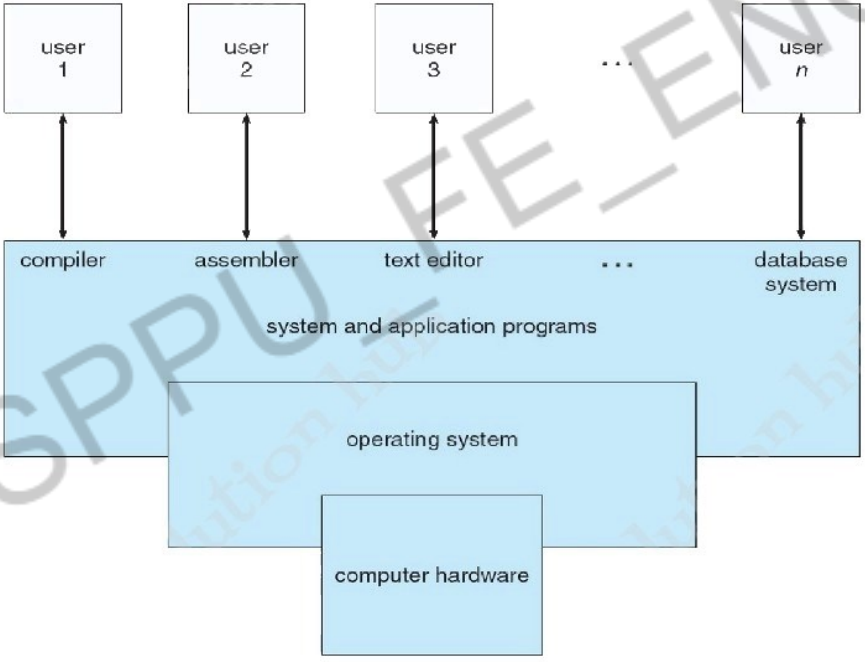
Operating System

An **Operating System** is a System software that manages all the resources of the computing device.

- Acts as an interface between the software and different parts of the computer or the computer hardware.
- Manages the overall resources and operations of the computer.
- Controls and monitors the execution of all other programs that reside in the computer, which also includes application programs and other system software of the computer.
- Examples of Operating Systems are Windows, Linux, macOS, Android, iOS, etc.

TG:SPPU_FE_ENGG

Abstract view of the components of a computer system



TG: SPPU_FE_ENGG

Objectives of OS

- **Convenient to use:** One of the objectives is to make the computer system more convenient to use in an efficient manner.
- **User Friendly:** To make the computer system more interactive with a more convenient interface for the users.
- **Easy Access:** To provide easy access to users for using resources by acting as an intermediary between the hardware and its users**.**
- **Management of Resources:** For managing the resources of a computer in a better and faster way.
- **Controls and Monitoring:** By keeping track of who is using which resource, granting resource requests, and mediating conflicting requests from different programs and users.
- **Fair Sharing of Resources:** Providing efficient and fair sharing of resources between the users and programs.

TOPPU_FE_ENGG

Functions of Operating System

- **Resource Management:** The operating system manages and allocates memory, CPU time, and other hardware resources among the various programs and processes running on the computer.
- **Process Management:** The operating system is responsible for starting, stopping, and managing processes and programs. It also controls the scheduling of processes and allocates resources to them.
- **Memory Management:** The operating system manages the computer's primary memory and provides mechanisms for optimizing memory usage.
- **Security:** The operating system provides a secure environment for the user, applications, and data by implementing security policies and mechanisms such as access controls and encryption.

TE SPPU - E&TC - ENGG

Functions of Operating System

- **Job Accounting:** It keeps track of time and resources used by various jobs or users.
- **File Management:** The operating system is responsible for organizing and managing the file system, including the creation, deletion, and manipulation of files and directories.
- **Device Management:** The operating system manages input/output devices such as printers, keyboards, mice, and displays. It provides the necessary drivers and interfaces to enable communication between the devices and the computer.
- **Networking:** The operating system provides networking capabilities such as establishing and managing network connections, handling network protocols, and sharing resources such as printers and files over a network.

TG. SPPU_FE_ENGG

Functions of Operating System

- **User Interface**: The operating system provides a user interface that enables users to interact with the computer system. This can be a Graphical User Interface (GUI), a Command-Line Interface (CLI), or a combination of both.
- **Backup and Recovery:** The operating system provides mechanisms for backing up data and recovering it in case of system failures, errors, or disasters.
- **Virtualization:** The operating system provides virtualization capabilities that allow multiple operating systems or applications to run on a single physical machine. This can enable efficient use of resources and flexibility in managing workloads.
- **Performance Monitoring:** The operating system provides tools for monitoring and optimizing system performance, including identifying bottlenecks, optimizing resource usage, and analyzing system logs and metrics.

TG. DPU FE ENGG

Functions of Operating System

- **Time-Sharing:** The operating system enables multiple users to share a computer system and its resources simultaneously by providing time-sharing mechanisms that allocate resources fairly and efficiently.
- **System Calls:** The operating system provides a set of system calls that enable applications to interact with the operating system and access its resources. System calls provide a standardized interface between applications and the operating system, enabling portability and compatibility across different hardware and software platforms.
- **Error-detecting Aids:** These contain methods that include the error messages, and other debugging and error-detecting methods.

TG:SPPU_FE_ENGG

Evolution of Operating System

- **1940s-1950s: Early Beginnings**
  - Computers operated without operating systems (OS).
  - Programs were manually loaded and run, one at a time.
  - The first operating system was introduced in 1956. It was a batch processing system GM-NAA I/O (1956) that automated job handling.
- **1960s: Multiprogramming and Timesharing**
  - Introduction of multiprogramming to utilize CPU efficiently.
  - Timesharing systems, like CTSS (1961) and Multics (1969), allowed multiple users to interact with a single system.
- **1970s: Unix and Personal Computers**
  - Unix (1971) revolutionized OS design with simplicity, portability, and multitasking.
  - Personal computers emerged, leading to simpler OSs like CP/M (1974) and PC-DOS (1981).

TE-SEM I-E_ENGG

Evolution of Operating System

- **1980s: GUI and Networking**
  - Graphical User Interfaces (GUIs) gained popularity with systems like Apple Macintosh (1984) and Microsoft Windows (1985).
  - Networking features, like TCP/IP in Unix, became essential.
- **1990s: Linux and Advanced GUIs**
  - Linux (1991) introduced open-source development.
  - Windows and Mac OS refined GUIs and gained widespread adoption.
- **2000s-Present: Mobility and Cloud**
  - Mobile OSs like iOS (2007) and Android (2008) dominate.
  - Cloud-based and virtualization technologies reshape computing, with OSs like Windows Server and Linux driving innovation.

TG. CSPU_FE_ENGG

Types of Operating System

- **Batch Operating System:** A Batch Operating System is a type of operating system that does not interact with the computer directly. There is an operator who takes similar jobs having the same requirements and groups them into batches.
- **Time-sharing Operating System:** Time-sharing Operating System is a type of operating system that allows many users to share computer resources (maximum utilization of the resources).
- **Distributed Operating System:** Distributed Operating System is a type of operating system that manages a group of different computers and makes appear to be a single computer. These operating systems are designed to operate on a network of computers. They allow multiple users to access shared resources and communicate with each other over the network. Examples include Microsoft Windows Server and various distributions of Linux designed for servers.

TG: SPPU_FE_ENGG

Types of Operating System

- **Network Operating System:** Network Operating System is a type of operating system that runs on a server and provides the capability to manage data, users, groups, security, applications, and other networking functions.
- **Real-time Operating System:** Real-time Operating System is a type of operating system that serves a real-time system and the time interval required to process and respond to inputs is very small. These operating systems are designed to respond to events in real time. They are used in applications that require quick and deterministic responses, such as embedded systems, industrial control systems, and robotics.
- **Multiprocessing Operating System:** Multiprocessor Operating Systems are used in operating systems to boost the performance of multiple CPUs within a single computer system. Multiple CPUs are linked together so that a job can be divided and executed more quickly.

TG.SPPU.E.ENGG

Types of Operating System

- **Single-User Operating Systems:** Single-User Operating Systems are designed to support a single user at a time. Examples include Microsoft Windows for personal computers and Apple macOS.
- **Multi-User Operating Systems:** Multi-User Operating Systems are designed to support multiple users simultaneously. Examples include Linux and Unix.
- **Embedded Operating Systems:** Embedded Operating Systems are designed to run on devices with limited resources, such as smartphones, wearable devices, and household appliances. Examples include Google's Android and Apple's iOS.
- **Cluster Operating Systems:** Cluster Operating Systems are designed to run on a group of computers, or a cluster, to work together as a single system. They are used for high-performance computing and for applications that require high availability and reliability. Examples include Rocks Cluster Distribution and OpenMPI.

TG-SPPU FE ENGG

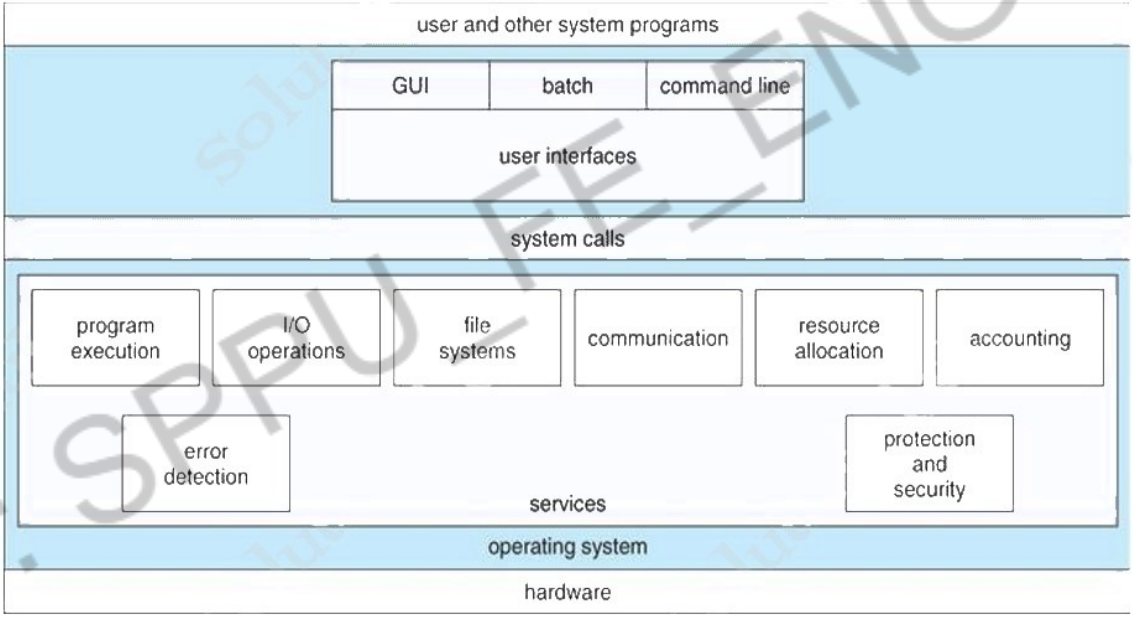
Operating System Services

An Operating System provides services to both the users and to the programs.

- It provides programs an environment to execute.
- It provides users the services to execute the programs in a convenient manner.

TG: SPPU_FE_ENGG

Operating System Services



TG: SPPU FE_ENGG

Operating System Services

- User Interface
- Program Execution
- I/O Operations
- File System manipulation
- Communication
- Error Detection
- Resource allocation
- Accounting
- Protection and Security.

TG. SPPU_FE_ENGG

User Interface

- User interface is essential and all operating systems provide it.
- Users either interacts with the operating system through the command-line interface or graphical user interface or GUI.
- The command interpreter executes the next user-specified command.
- A GUI offers the user a mouse-based window and menu system as an interface.

TG: SPPU_FE_ENGG

Program Execution

Operating systems handle many kinds of activities from user programs to system programs like printer spooler, name servers, file server, etc. Each of these activities is encapsulated as a process.

A process includes the complete execution context (code to execute, data to manipulate, registers, OS resources in use). Following are the major activities of an operating system with respect to program management −

- Loads a program into memory.
- Executes the program.
- Handles program's execution.
- Provides a mechanism for process synchronization.
- Provides a mechanism for process communication.
- Provides a mechanism for deadlock handling.

TG:SPPU_FE_ENGG

I/O Operation

An I/O subsystem comprises of I/O devices and their corresponding driver software. Drivers hide the peculiarities of specific hardware devices from the users.

An Operating System manages the communication between user and device drivers.

- I/O operation means read or write operation with any file or any specific I/O device.
- Operating system provides the access to the required I/O device when required.

TG: SPPU_FE_ENGG

File System Manipulation

- A file represents a collection of related information. Computers can store files on the disk (secondary storage), for long-term storage purpose. Examples of storage media include magnetic tape, magnetic disk and optical disk drives like CD, DVD. Each of these media has its own properties like speed, capacity, data transfer rate and data access methods.
- A file system is normally organized into directories for easy navigation and usage. These directories may contain files and other directions. Following are the major activities of an operating system with respect to file management −
- Program needs to read a file or write a file.
- The operating system gives the permission to the program for operation on file.
- Permission varies from read-only, read-write, denied and so on.
- Operating System provides an interface to the user to create/delete files.
- Operating System provides an interface to the user to create/delete directories.
- Operating System provides an interface to create the backup of file system.

TCS SPPU TE ELEC

Communication

- In case of distributed systems which are a collection of processors that do not share memory, peripheral devices, or a clock, the operating system manages communications between all the processes. Multiple processes communicate with one another through communication lines in the network.
- The OS handles routing and connection strategies, and the problems of contention and security. Following are the major activities of an operating system with respect to communication –
- Two processes often require data to be transferred between them
- Both the processes can be on one computer or on different computers, but are connected through a computer network.
- Communication may be implemented by two methods, either by Shared Memory or by Message Passing.

TC-SPPU-FE-ENGG

Error Detection

Errors can occur anytime and anywhere. An error may occur in CPU, in I/O devices or in the memory hardware. Following are the major activities of an operating system with respect to error handling −

- The OS constantly checks for possible errors.
- The OS takes an appropriate action to ensure correct and consistent computing.

TG: SPPU_FE_ENGG

Resource Allcation

- In case of multi-user or multi-tasking environment, resources such as main memory, CPU cycles and files storage are to be allocated to each user or job. Following are the major activities of an operating system with respect to resource management –
- The OS manages all kinds of resources using schedulers.
- CPU scheduling algorithms are used for better utilization of CPU.

TG: SPPU_FE_ENGG

Accounting

- How much use of Computer Resources
- Which Resources

TG: SPPU_FE_ENGG

Protection and Security

- Considering a computer system having multiple users and concurrent execution of multiple processes, the various processes must be protected from each other's activities.
- Protection refers to a mechanism or a way to control the access of programs, processes, or users to the resources defined by a computer system. Following are the major activities of an operating system with respect to protection −
- The OS ensures that all access to system resources is controlled.
- The OS ensures that external I/O devices are protected from invalid access attempts.
- The OS provides authentication features for each user by means of passwords.

TG. SPPU - TE - ENGG

System Calls

- A number of services are requested by the program, and the OS responds by launching a number of systems calls to fulfill the request.
- A system call can be written in high-level languages like C or Pascal or in assembly language. If a high-level language is used, the operating system may directly invoke system calls, which are predefined functions.
- A system call is initiated by the program executing a specific instruction, which triggers a switch to kernel mode, allowing the program to request a service from the OS. The OS then handles the request, performs the necessary operations, and returns the result back to the program.
- System calls are essential for the proper functioning of an operating system, as they provide a standardized way for programs to access system resources. Without system calls, each program would need to implement its methods for accessing hardware and system services, leading to inconsistent and error-prone behavior.
- A number of services are requested by the program, and the OS responds by launching a number of systems calls to fulfill the request.
- A system call can be written in high-level languages like C or Pascal or in assembly language. If a high-level language is used, the operating system may directly invoke system calls, which are predefined functions.
- A system call is initiated by the program executing a specific instruction, which triggers a switch to kernel mode, allowing the program to request a service from the OS. The OS then handles the request, performs the necessary operations, and returns the result back to the program.
- System calls are essential for the proper functioning of an operating system, as they provide a standardized way for programs to access system resources. Without system calls, each program would need to implement its methods for accessing hardware and system services, leading to inconsistent and error-prone behavior.

TE-CSPU-FE-ENGG

System Calls

- **Services Provided by System Calls**
- Process Creation and Management
- Main Memory Management
- File Access, Directory, and File System Management
- Device Handling(I/O)
- Protection
- Networking, etc.
  - **Process Control:** end, abort, create, terminate, allocate, and free memory.
  - **File Management:** create, open, close, delete, read files, etc.
  - **Device Management**
  - **Information Maintenance**
  - **Communication**

TG:SPPU_FE_ENGG

System Calls

- System Calls act as a bridge between an operating system (OS) and a running program. They are usually written as assembly language instructions and are detailed in manuals for programmers working with assembly language.
- When a program running in user mode needs to access a resource, it makes a System Call. This request is sent to the OS kernel to obtain the needed resource.
- System Calls are used for various tasks, such as:
- Creating or executing files in the file system.
- Reading from and writing to files.
- Developing and managing new procedures in programs.
- Making network connections, including sending and receiving data packets.
- Accessing hardware devices like printers and scanners.
-

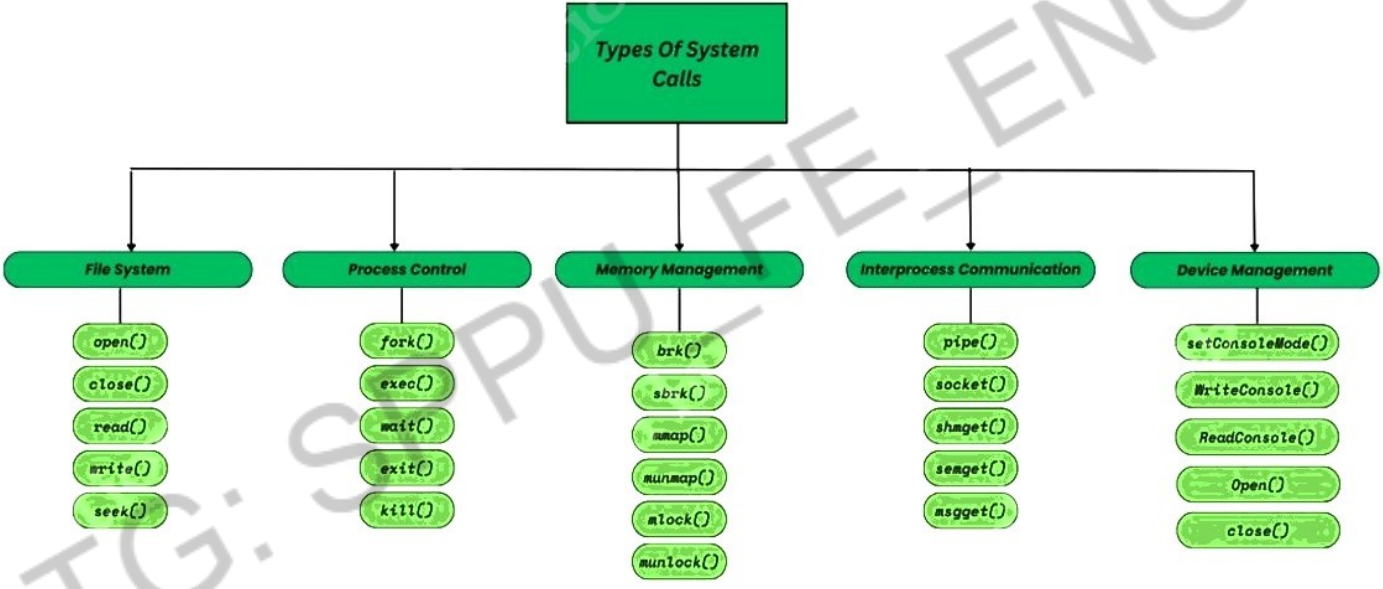
TG:SPPU_FE_ENGG

Features of System Calls

- **Interface:** System calls provide a well-defined interface between user programs and the operating system. Programs make requests by calling specific functions, and the operating system responds by executing the requested service and returning a result.
- **Protection:** System calls are used to access privileged operations that are not available to normal user programs. The operating system uses this privilege to protect the system from malicious or unauthorized access.
- **Kernel Mode:** When a system call is made, the program is temporarily switched from user mode to kernel mode. In kernel mode, the program has access to all system resources, including hardware, memory, and other processes.
- **Context Switching:** A system call requires a context switch, which involves saving the state of the current process and switching to the kernel mode to execute the requested service. This can introduce overhead, which can impact system performance.
- **Error Handling:** System calls can return error codes to indicate problems with the requested service. Programs must check for these errors and handle them appropriately.
- **Synchronization:** System calls can be used to synchronize access to shared resources, such as files or network connections. The operating system provides synchronization mechanisms, such as locks or semaphores, to ensure that multiple programs can access these resources safely.

TG.DPU.LIFE.ENGG

Types of System Calls



TG: SPPU FE ENGG

Types of System Calls

- **Process Control**- These system calls deal with processes such as process creation, process termination etc.
- **File Management**-These system calls are responsible for file manipulation such as creating a file, reading a file, writing into a file etc.
- **Device Management**- These system calls are responsible for device manipulation such as reading from device buffers, writing into device buffers etc.
- **Information Maintenance**- These system calls handle information and its transfer between the operating system and the user program.
- **Communication**- These system calls are useful for interprocess communication. They also deal with creating and deleting a communication connection.

TG:SPPU_FE_ENGG

Types of System Calls

Types of System Calls	Windows	Linux
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Management	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Management	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()

TG: SPPU_FE_ENGG

Types of System Calls

1. File System Operations

- These system calls are made while working with files in OS, File manipulation operations such as creation, deletion, termination etc.
- **open():** Opens a file for reading or writing. A file could be of any type like text file, audio file etc.
- **read():** Reads data from a file. Just after the file is opened through open() system call, then if some process want to read the data from a file, then it will make a read() system call.
- **write():** Writes data to a file. Wheneve the user makes any kind of modification in a file and saves it, that's when this is called.
- **close():** Closes a previously opened file.
- **seek():** Moves the file pointer within a file. This call is typically made when we the user tries to read the data from a specific position in a file. For example, read from line - 47. Than the file pointer will move from line 1 or wherever it was previously to line-47.

TGTSPU_FE_ENGG

Types of System Calls

2. Process Control

- These types of system calls deal with process creation, process termination, process allocation, deallocation etc. Basically manages all the process that are a part of OS.
- **fork():** Creates a new process (child) by duplicating the current process (parent). This call is made when a process makes a copy of itself and the parent process is halted temporarily until the child process finishes its execution.
- **exec():** Loads and runs a new program in the current process and replaces the current process with a new process. All the data such as stack, register, heap memory everything is replaced by a new process and this is known as overlay. For example, when you execute a java byte code using command - java "filename". Then in the background, exec() call will be made to execute the java file and JVM will also be executed.
- **wait():** The primary purpose of this call is to ensure that the parent process doesn't proceed further with its execution until all its child processes have finished their execution. This call is made when one or more child processes are forked.
- **exit()**: It simply terminates the current process.
- **kill():** This call sends a signal to a specific process and has various purpose including - requesting it to quit voluntarily, or force quit, or reload configuration.

TY_SPRUCE_ENGG

Types of System Calls

3. Memory Management

- These types of system calls deals with memory allocation, deallocation & dynamically changing the size of a memory allocated to a process. In short, the overall management of memory is done by making these system calls.
- **brk():** Changes the data segment size for a process in HEAP Memory. It takes an address as argument to define the end of the heap and explicitly sets the size of HEAP.
- **sbrk():** This call is also for memory management in heap, it also takes an argument as an integer (+ve or -ve) specifying whether to increase or decrease the size respectively.
- **mmap(): Memory** Map - It basically maps a file or device into main memory and further into a process's address space for performing operations. And any changes made in the content of a file will be reflected in the actual file.
- **munmap():** Unmaps a memory-mapped file from a process's address space and out of main memory
- **mlock() and unlock():** memory lock defines a mechanism through which certain pages stay in memory and are not swapped out to the swap space in the disk. This could be done to avoid page faults. Memory unlock is the opposite of lock, it releases the lock previously acquired on pages.

TY SPPU E-ENGG

Types of System Calls

4. Interprocess Communication (IPC)

- When two or more process are required to communicate, then various IPC mechanism are used by the OS which involves making numerous system calls. Some of them are :
- **pipe():** Creates a unidirectional communication channel between processes. For example, a parent process may communicate to its child process through a pipe making a parent process as input source of its child process.
- **socket():** Creates a network socket for communication. Processes in same or other networks can communicate through this socket, provided that they have necessary network permissions granted.
- **shmget():** It is short for - 'shared-memory-get'. It allows one or more processes to share a portion of memory and achieve interprocess communication.
- **semget():** It is short for - 'semaphore-get'. This call typically manages the coordination of multiple processes while accessing a shared resource that is, the critical section.
- **msgget():** It is short for - 'message-get'. IPC mechanism has one of the fundamental concept called - 'message queue' which is a queue data structure inside memory through which various processes communicate with each other. This message queue is allocated through this call allowing other processes a structured way of communication for data exchange purpose.

TG. DEPT_ENGG

Types of System Calls

5. Device Management

- The device management system calls are used to interact with various peripherial devices attached to the PC or even the management of the current device.
- **SetConsoleMode():** This call is made to set the mode of console (input or output). It allows a process to control various console modes. In windows, it is used to control the behaviour of command line.
- **WriteConsole():** It allows us to write data on console screen.
- **ReadConsole():** It allows us to read data from console screen (if any arguments are provided).
- **open():** This call is made whenever a device or a file is opened. A unique file descriptor is created to maintain the control access to the opened file or device.
- **close():** This call is made when the system or the user closes the file or device.

TC-SPPU_FE_ENGG

Types of System Calls

Importance of System Calls

- **Efficient Resource Management:** System Calls help your computer manage its resources efficiently. They allocate and manage memory so programs run smoothly without using up too many resources. This is important for multitasking and overall performance.
- **Security and Isolation:** System Calls ensure that one program cannot interfere with or access the memory of another program. This enhances the security and stability of your device.
- **Multitasking Capabilities:** System Calls support multitasking, allowing multiple programs to run simultaneously. This improves productivity and makes it easy to switch between applications.
- **Enhanced Control:** System Calls provide a high level of control over your device's operations. They allow you to start and stop processes, manage files, and perform various system-related tasks.
- **Input/Output (I/O) Operations:** System Calls enable communication with input and output devices, such as your keyboard, mouse, and screen. They ensure that these devices work effectively.
- **Networking and Communication:** System Calls facilitate networking and communication between different applications. They make it easy to transfer data over networks, browse the web, send emails, and connect online.

TG. SEM - IT - ENGG

Types of System Calls

Types of System Calls	Windows	Linux
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Management	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Management	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()

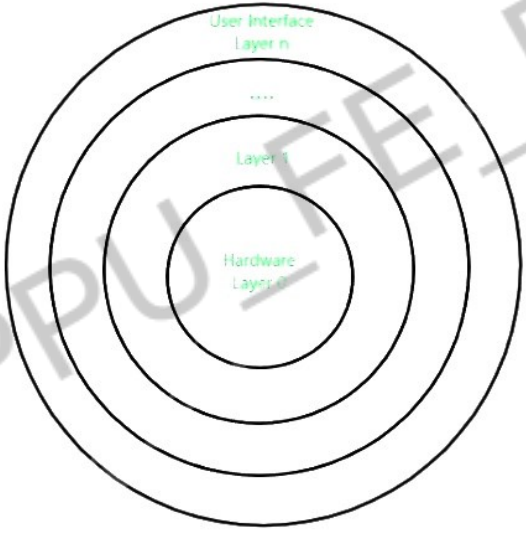
TG: SPPU_FE_ENGG

Operating System Structure: Layered Approach

- The different services of the operating system are split into various layers
- Each layer has a specific well-defined task to perform
- It was created to improve the pre-existing structures like the Monolithic structure (UNIX) and the Simple structure (MS-DOS)
- **Example -** The Windows NT operating system uses this layered approach as a part of it.
- **Design Analysis :** The whole Operating System is separated into several layers (from 0 to n) as the diagram shows. Each of the layers must have its own specific function to perform. There are some rules in the implementation of the layers as follows.
- The outermost layer must be the User Interface layer.
- The innermost layer must be the Hardware layer.
- A particular layer can access all the layers present below it but it cannot access the layers present above it. That is layer n-1 can access all the layers from n-2 to 0 but it cannot access the nth layer.
- Thus if the user layer wants to interact with the hardware layer, the response will be traveled through all the layers from n-1 to 1. Each layer must be designed and implemented such that it will need only the services provided by the layers below it.

TY_SPPU_FE_ENGG

Operating System Structure: Layered Approach



TG: SPPU_FE_ENGG

Operating System Structure: Layered Approach

- **Advantages :**
- There are several advantages to this design :
- **Modularity :** This design promotes modularity as each layer performs only the tasks it is scheduled to perform.
- **Easy debugging :** As the layers are discrete so it is very easy to debug. Suppose an error occurs in the CPU scheduling layer, so the developer can only search that particular layer to debug, unlike the Monolithic system in which all the services are present together.
- **Easy update :** A modification made in a particular layer will not affect the other layers.
- **No direct access to hardware :** The hardware layer is the innermost layer present in the design. So a user can use the services of hardware but cannot directly modify or access it, unlike the Simple system in which the user had direct access to the hardware.
- **Abstraction :** Every layer is concerned with its own functions. So the functions and implementations of the other layers are abstract to it.

TG: SE IT ENGG

Operating System Structure: Layered Approach

- **Disadvantages :**
- Though this system has several advantages over the Monolithic and Simple design, there are also some disadvantages as follows.
- **Complex and careful implementation :** As a layer can access the services of the layers below it, so the arrangement of the layers must be done carefully. For example, the backing storage layer uses the services of the memory management layer. So it must be kept below the memory management layer. Thus with great modularity comes complex implementation.
- **Slower in execution :** If a layer wants to interact with another layer, it sends a request that has to travel through all the layers present in between the two interacting layers. Thus it increases response time, unlike the Monolithic system which is faster than this. Thus an increase in the number of layers may lead to a very inefficient design.

TG.SEPU_IT_ENGG

Monolithic Approach

- a very simple operating system where the kernel directly controls device management, memory management, file management, and process management.
- All of the system's resources are accessible to the kernel.
- Every part of the operating system is contained within the kernel in monolithic systems. Monolithic architecture-based operating systems were initially introduced in the 1970s.
- All hardware components are managed by the monolithic kernel, which functions as a virtual machine.
- Unlike a microkernel, which has fewer tasks, it is not the same. The two sections of a microkernel are called kernel space and user space. IPC (inter-process communication) is the means by which both components speak with one another. One benefit of microkernel is that in the event of a failure, the other server assumes control.

TG: SPPU_FE_ENGG

Microkernel Operating System

A MicroKernel is an approach to designing an Operating System (OS). The microkernel provides very fundamental services required to run the OS like basic memory management, task scheduling, etc. Microkernels use **Inter-Process Communication (IPC)** for communication. In this article, we will discuss Kernels, Microkernel, its architecture, etc.

- **What is Kernel?**
- A Kernel is the core part of an operating system that manages system resources. It also acts as a bridge between the application and hardware of the computer. It is one of the first programs loaded on start-up (after the Bootloader).
- A Microkernel is one of the classifications of the kernel. Being a kernel it manages all system resources. But in a microkernel, the **user services** and **kernel services** are implemented in different address spaces. The user services are kept in **user address space**, and kernel services are kept under **kernel address space**, thus also reducing the size of the kernel and the size of the operating system as well.

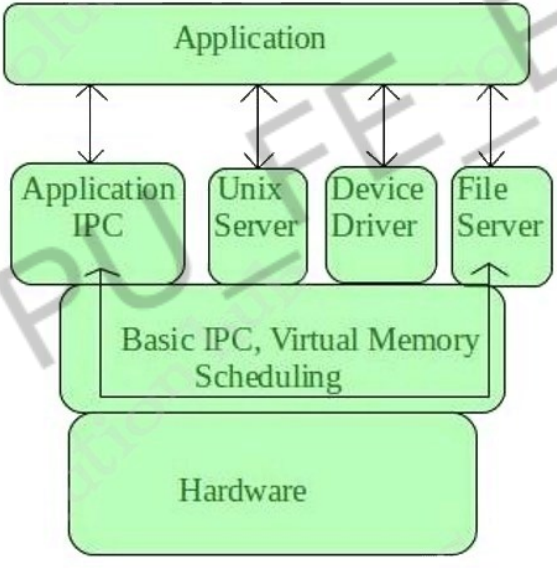
TG. SPPU_FE_ENGG

What is Microkernel?

- Microkernel is a type of Operating System that provides some basic services for an operating system/ These services include memory management, process scheduling, etc. Some other services like Device Drivers, File Systems, etc are managed by user-level processes. User Level Process communicates with Microkernel via message passing. This way of handling the process makes microkernels more modular and more flexible than traditional monolithic kernels.
- The main advantage of a microkernel architecture is that it provides a more secure and stable operating system. Since only the most essential services run in kernel space, the attack surface of the operating system is reduced, making it more difficult for an attacker to exploit vulnerabilities. The main disadvantage of microkernel is that message passing between user-level processes can be slower than direct system calls in a monolithic kernel. This can affect the performance of the operating system, especially in high-performance applications.

TG: SPPU_E_ENGG

Architecture of Microkernel Operating System



TG: SPPU_FE_ENGG

Features of Microkernel Operating System

- **Minimal Core** : The kernel only manages basic tasks like CPU scheduling, memory management, and inter-process communication. All other services (like device drivers) run in user space, outside the kernel.
- **Modularity** : System services are separated into independent modules. This makes it easy to update or replace individual components without affecting the entire system.
- **Stability and Fault Isolation** : If a service like a device driver fails, it won't crash the whole system. Each service runs independently, so the core OS remains stable.
- **Ease of Maintenance** : Since services are independent, fixing bugs or adding new features doesn't require major changes to the kernel itself, making maintenance easier.

TG: SPPU_FE_ENGG

Advantages of Microkernel Operating System

- **Modularity:** Because the kernel and servers can be developed and maintained independently, the microkernel design allows for greater modularity. This can make adding and removing features and services from the system easier.
- **Performance:** Because the kernel only contains the essential functions required to manage the system, the microkernel design can improve performance. This can make the system faster and more efficient.
- **Security:** The microkernel design can improve security by reducing the system's attack surface by limiting the functions provided by the kernel. Malicious software may find it more difficult to compromise the system as a result of this.
- **Reliability:** Microkernels are less complex than monolithic kernels, which can make them more reliable and less prone to crashes or other issues.
- **Scalability:** Microkernels can be easily scaled to support different hardware architectures, making them more versatile.
- **Portability:** Microkernels can be ported to different platforms with minimal effort, which makes them useful for embedded systems and other specialized applications.

TG:SPPU_TE_ENGG

Disadvantages of Microkernel Operating System

- **Slower Message Passing:** Slower message passing between user-level processes can affect performance, especially in high-performance applications.
- **More Complex:** Increased complexity due to the modular design can make it more difficult to develop and maintain the operating system .
- **Limited Performance:** Limited performance optimization due to separation of kernel and user-level processes.
- **Higher Memory Usage:** Higher memory usage compared to a monolithic kernel.

TG: SPPU_FE_ENGG

Introduction to Linux OS:

- Linux is based on the **UNIX** operating system. **UNIX** is a powerful, multi-user, multitasking operating system originally developed in the **1970s at AT&T Bell Labs**. It laid the foundation for many modern operating systems, including Linux.
- Linux is free and open-source, accessible to everyone.
- Its source code can be inspected and modified by anyone.
- This promotes global collaboration and innovation.
- Linux offers efficient performance and strong security.
- It works well across many devices and industries.

TG. SPPU_FE_ENGG

Introduction to Linux OS:

- The Linux Operating System is a Unix-like operating system that combines a wide range of open-source tools and components to form a complete computing environment.
- These components include file systems, user interfaces, system utilities, and application programs all working together to manage hardware and enable users to interact with their computer systems.
- To create a full and functional system, the Linux is combined with a collection of software packages and utilities, which are together called Linux distributions.

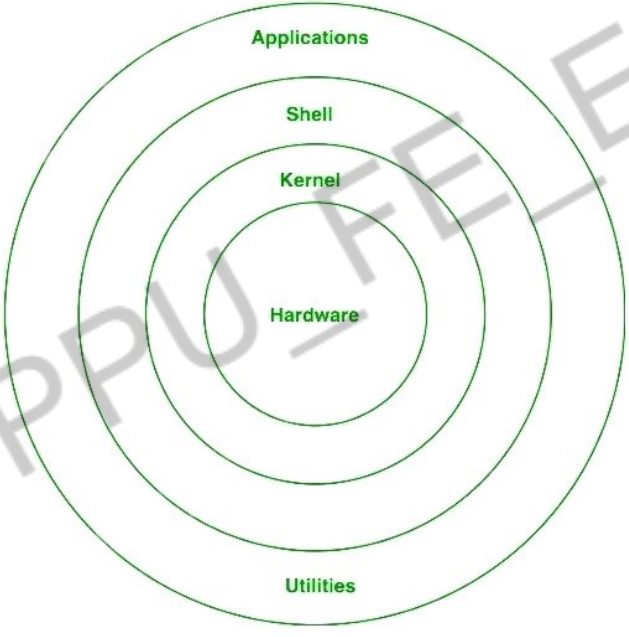
TG: SPPU_FE_ENGG

Why to Use Linux OS?

- Because it is free, open-source, and extremely flexible, Linux is widely utilized.
- For servers and developers, it is the ideal option because it offers strong security, stability, and performance.
- Generally interoperable hardware, a broad software library, and a vibrant community that offers support and regular updates are the many benefits of Linux.
- Due to its adaptability, users can customize the operating system according to their own needs, whether they become for personal or large enterprise use.

TG:SPPU_FE_ENGG

Components of Linux System:



TG: SPPU_FE_ENGG

Components of Linux System:

1. Kernel: Kernel is the core of the Linux based operating system. It virtualizes the common hardware resources of the computer to provide each process with its virtual resources. This makes the process seem as if it is the sole process running on the machine. The kernel is also responsible for preventing and mitigating conflicts between different processes. Different types of the kernel are:

- Monolithic Kernel
- Hybrid kernels
- Exo kernels
- Micro kernels

2. System Library: Linux uses system libraries, also known as shared libraries, to implement various functionalities of the operating system. These libraries contain pre-written code that applications can use to perform specific tasks. By using these libraries, developers can save time and effort, as they don't need to write the same code repeatedly. System libraries act as an interface between applications and the kernel, providing a standardized and efficient way for applications to interact with the underlying system.

TG. SPPU_FE_ENGG

Components of Linux System:

3. Shell: The shell is the user interface of the Linux Operating System. It allows users to interact with the system by entering commands, which the shell interprets and executes. The shell serves as a bridge between the user and the kernel, forwarding the user's requests to the kernel for processing. It provides a convenient way for users to perform various tasks, such as running programs, managing files, and configuring the system.

4. Hardware Layer: The hardware layer encompasses all the physical components of the computer, such as RAM (Random Access Memory), HDD (Hard Disk Drive), CPU (Central Processing Unit), and input/output devices. This layer is responsible for interacting with the Linux Operating System and providing the necessary resources for the system and applications to function properly. The Linux kernel and system libraries enable communication and control over these hardware components, ensuring that they work harmoniously together.

TOSHNI PENGG

Components of Linux System:

5. System Utility: System utilities are essential tools and programs provided by the Linux Operating System to manage and configure various aspects of the system.

-These utilities perform tasks such as installing software, configuring network settings, monitoring system performance, managing users and permissions, and much more.

-System utilities simplify system administration tasks, making it easier for users to maintain their Linux systems efficiently.

TG: SPPU_FE_ENGG

Uses of Linux Operating System:

- **Servers and Hosting**: Powers web servers, cloud infrastructure, and database management systems.
- **Development**: Used by developers for coding, debugging, and running applications.
- **Desktop and Personal Use**: Provides secure and customizable desktop environments.
- **Cybersecurity**: Essential for ethical hacking, penetration testing, and security research.
- **Embedded Systems**: Runs lightweight devices like routers, IoT gadgets, and smart appliances.
- **Supercomputers**: Dominates high-performance computing for scientific research and simulations.
- **Education**: A cost-effective tool for teaching programming and system administration.

TG: SPPU_FE_ENGG

Basic Shell Commands:

A shell **in Linux** is a program that serves as an interface between the **user and the operating system.** It accepts commands from the user, interprets them, and passes them to the operating system for execution. The commands can be used for a wide range of tasks, from **file manipulation** to **system management**.

- Some of the essential **basic shell commands** in **Linux** for different operations are:
- **File Management** -> cp, mv, rm, mkdir
- **Navigation** -> cd, pwd, ls
- **Text Processing** -> cat, grep, sort, head
- **System Monitoring** -> top, ps, df
- **Permissions and Ownership** -> chmod, chown, chgrp
- **Networking** - > ping, wget, curl, ssh, scp, ftp
- **Compression and Archiving** - > tar, gzip, gunzip, zip, unzip
- **Package Management** - > dnf, yum, apt-get
- **Process Management** -> kill, killall, bg, killall, kill

TE_SPPU_FE_ENGG

Basic Shell Commands for File & Directory Management

A shell

Command	Description	Example
ls	Lists files and directories	ls
cd	Changes the current directory	cd /home/user/Documents
pwd	Displays the current directory path	pwd
mkdir	Creates a new directory	mkdir new_directory
rm	Removes files or directories	rm file.txt
cp	Copies files or directories	cp file1.txt file2.txt
mv	Moves or renames files and directories	mv old_name new_name
touch	Creates an empty file or updates file timestamps	touch newfile.txt

TG: SPPU_FE_ENGG



CLICK HERE TO JOIN WHATSAPP CHANNEL FOR SPPU UNIVERSITY

👇 **JUST CLICK ON LINK** 👇

https://www.whatsapp.com/channel/0029VaIjFrIICVfpcV9HFc3b

TG:SPPU_FE_ENGG